BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE APPLICATIONS AND GEO-INFORMATICS

WEEKLY PROGRESS REPORT (19/10/2026 – 25/10/2026)

WEEK 13

PROJECT NAME	CHATBOT FOR STRUCTURED, UNSTRUCTURED AND SEMANTIC KNOWLEDGE QUERY

PROJECT DESCRIPTION :	DESIGN AND IMPLEMENT A CHATBOT THAT CAN ANSWER TO DIFFERENT TYPE OF QUERY LIKE STRUCTURED, UNSTRUCTURED AND SEMANTIC KNOWLEDGE BY DECIDING OWN WHAT TYPE OF QUERY IT IS
GROUP MEMBER :	PRATYUSH ANAND, ADITYA SINGH
GROUP ID :	2026G306
GROUP GUIDE :	VEDANT JOSHI
GROUP COORDINATOR :	SIDHDHARTH PATEL
COLLEGE GUIDE NAME :	EHT SHAM
COLLEGE GUIDE No. :	+91 9818938592
COLLEGE GUIDE EMAIL. :	Ehte.sham@pilani.bits-pilani.ac.in
COLLEGE NAME :	BITS PILANI

19/10/2026	• **Reviewed modern RAG techniques (hybrid search, reranking, semantic chunking, parent-child and contextual retrieval, Corrective RAG, HyDE, GraphRAG) against the chatbot and the 2-3 GB VRAM budget; shortlisted the ones that address measured failures.** • **Took a full snapshot of the knowledge-pool and organization-isolation work before starting the upgrade.**
20/10/2026	• **Integrated Unstructured.io parsing for PDF, DOCX, TXT, MD and HTML (titles, paragraphs, list items, tables with rows); fixed Windows issues (libmagic crash, missing unstructured-inference) and kept the old parsers as fallback. CSV/Excel stay complete SQL tables.**
21/10/2026	• **Implemented semantic chunking with parent-child retrieval: small chunks are searched and their parent section is given to the model, with document and section headers added to every chunk.** • **Added numbered-heading detection and a 1,500-character window so one long section cannot crowd out other evidence.**
22/10/2026	• **Implemented Corrective RAG with multi-query rewriting and HyDE, used only when evidence is missing or weak and skipped for off-topic questions (they still cost zero model calls).** • **Added protection against indirect prompt injection: instruction-like text inside uploaded documents is removed before it reaches the model.**
23/10/2026	• **Improved speed with model warm-up at server start and Ollama keep-alive, so the first answer no longer waits for model loading; added a re-index script that rebuilds an organization from its stored originals.** • **Added 28 automated tests (337 total, all passing), including a full guardrail matrix; real-model checks showed two reworded questions still missed, queued for next week.**
24/10/2026	**Holiday (4th Saturday)**
25/10/2026	**Holiday (Sunday)**

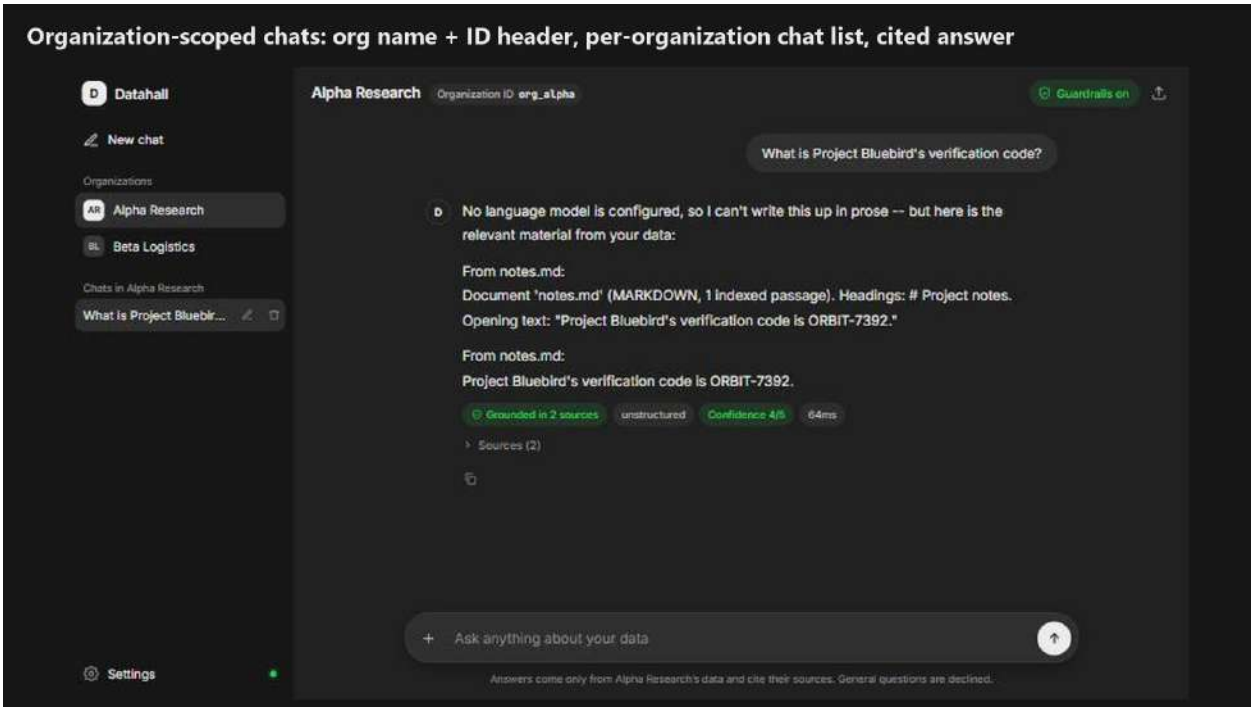
Next Week Plan	**Fix the remaining reworded-question retrieval misses, run the full**

	old-vs-new evaluation on real models (accuracy, citations, latency) and test the guardrails end to end with real documents.

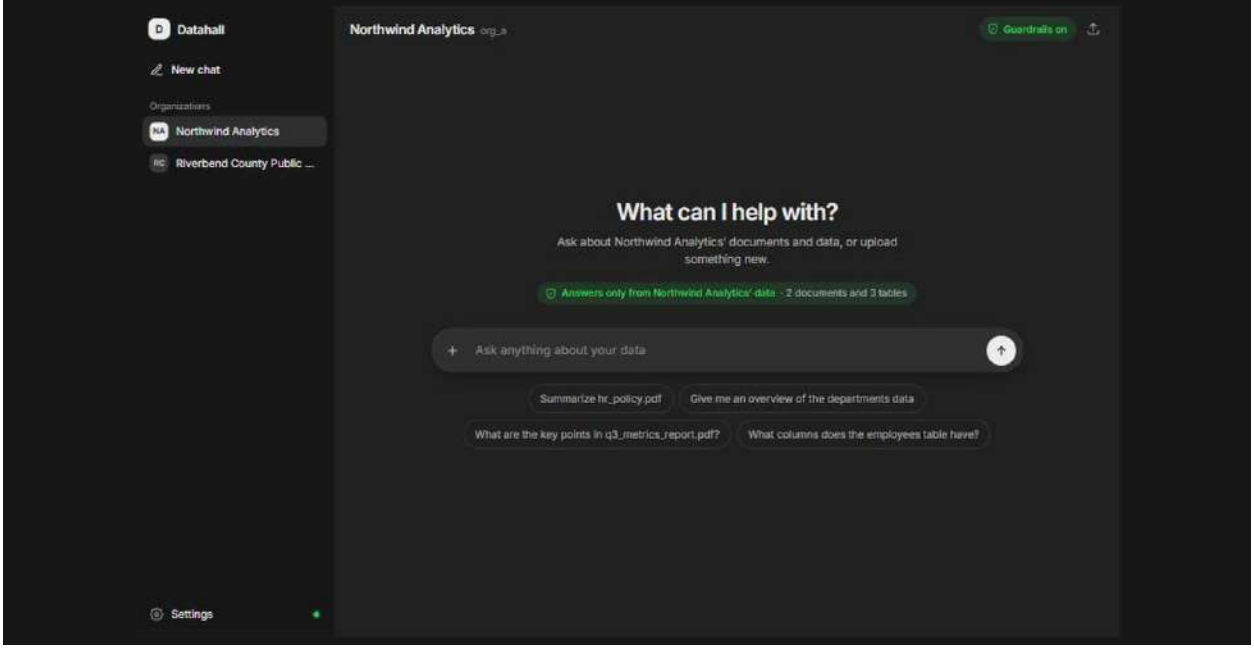
REFERENCE:

- https://docs.unstructured.io/open-source/introduction/overview
- https://www.anthropic.com/news/contextual-retrieval
- https://arxiv.org/abs/2401.15884
- https://arxiv.org/abs/2212.10496
- https://python.langchain.com/docs/how_to/parent_document_retriever/
- https://qdrant.tech/documentation/concepts/hybrid-queries/
- https://huggingface.co/BAAI/bge-small-en-v1.5

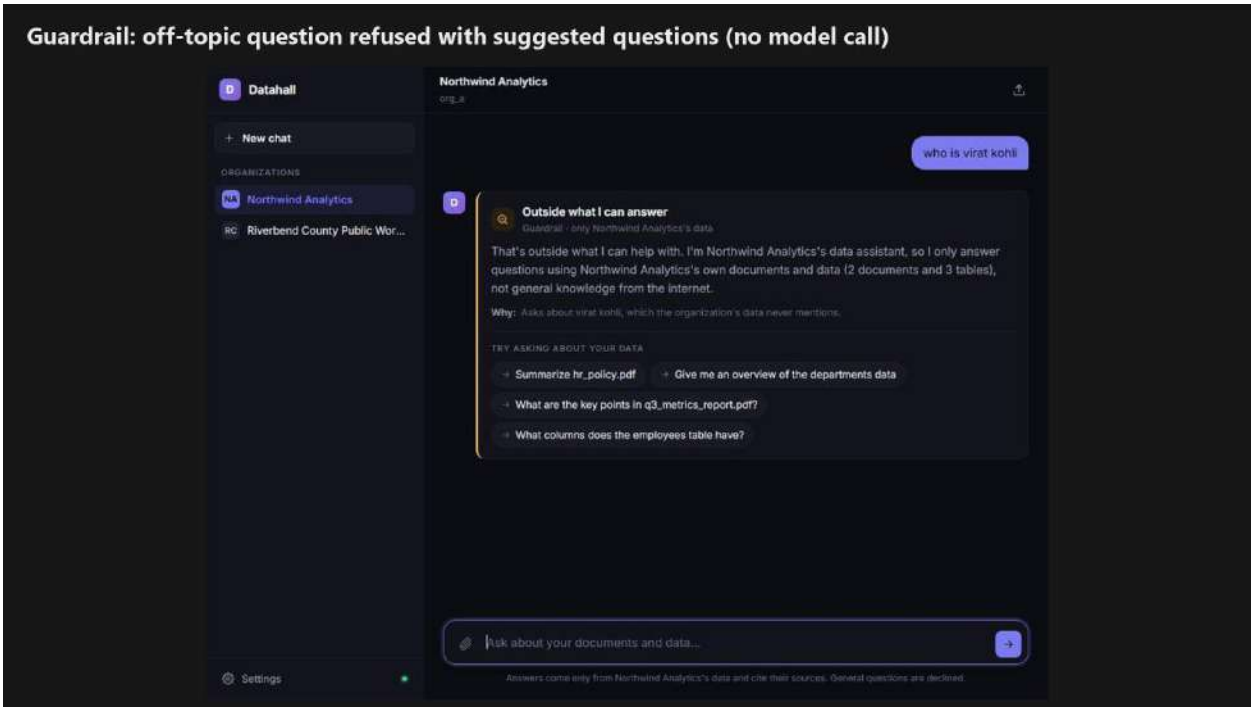
SCREEN SHOTS:



Redesigned start screen: scope banner and suggestions from the organization's own files



Guardrail: off-topic question refused with suggested questions (no model call)



Unstructured.io parsing mapped to typed blocks (titles, lists, tables)

```python
def load_with_unstructured(path: str | Path) -> ParsedDocument:
    path = Path(path)
    elements = _partition(path)
    blocks: list[ParsedBlock] = []
    for el in elements:
        kind = type(el).__name__
        text = (str(el) or "").strip()
        page = getattr(el.metadata, "page_number", None)
        if kind in _SKIP or (not text and kind != "Table"):
            continue
        if kind != "Table" and _NUMBERED_HEADING.match(text) and len(text.split()) <= 8:
            level = text.split()[0].rstrip(".").count(".") + 1
            blocks.append(ParsedBlock(type="heading", text=text, page_number=page, heading_level=level))
            continue
        if kind == "Title":
            words = len(text.split())
            if _SENTENCE_END.search(text) or words > 14:
                blocks.append(ParsedBlock(type="paragraph", text=text, page_number=page))
            else:
                depth = getattr(el.metadata, "category_depth", None)
                blocks.append(ParsedBlock(type="heading", text=text, page_number=page,
                                          heading_level=(depth + 1) if isinstance(depth, int) else 1))
        elif kind == "ListItem":
            blocks.append(ParsedBlock(type="list_item", text=text, page_number=page))
        elif kind == "Table":
            rows = _table_rows(el)
            if rows:
                blocks.append(ParsedBlock(type="table", text=text, page_number=page, table_rows=rows))
            elif text:
                blocks.append(ParsedBlock(type="paragraph", text=text, page_number=page))
        else:
            blocks.append(ParsedBlock(type="paragraph", text=text, page_number=page))

    source_type = {".pdf": "pdf", ".docx": "docx", ".md": "markdown", ".html": "html", ".htm": "html"}.get(path.suffix.lower(), "text")
```

Corrective RAG: multi-query rewriting + HyDE, skipped for off-topic questions

```python
def _corrective_search(state: AgentState, query: str, pool_dict: dict) -> dict:
    """Corrective retrieval (CRAG) with multi-query rewriting + HyDE: one model
    call, at most n+1 extra searches, one round. Never for questions that the
    organization's data plainly does not cover (they stay at 0 model calls)."""
    settings = get_settings()
    info: dict[str, Any] = {"triggered": False}
    pool_dict["correction"] = info
    if not settings.corrective_retrieval:
        info["reason"] = "disabled"
        return pool_dict
    client = get_llm_client()
    if is_noop_client(client):
        info["reason"] = "no language model"
        return pool_dict
    scope = assess_scope(state["user_query"], state["tenant_id"], MockLLMClient())
    if not scope.in_scope:
        info["reason"] = f"not attempted: {scope.rule}"
        return pool_dict
    history = state.get("conversation_history") or []
    history_text = ("Earlier conversation:\n" + "\n".join(f"{t.get('role')}: {t.get('content', '')[:200]}" for t in history[-2:]) + "\n") if history else ""
    try:
        result = client.generate_json(_REWRITE_PROMPT.format(query=state["user_query"], history=history_text,
                                                             n=settings.corrective_query_count))
    except Exception as exc:  # noqa: BLE001
        info.update(triggered=True, reason=f"rewrite failed: {exc}")
        return pool_dict
    queries = [str(q).strip() for q in (result.get("queries") or []) if str(q).strip()][: settings.corrective_query_count]
    passage = str(result.get("hypothetical_passage") or "").strip()
    searches = queries + ([passage] if passage else [])
    info.update(triggered=True, queries=queries, hypothetical_passage=passage[:400])
    if not searches:
        info["reason"] = "model returned no rewrites"
        return pool_dict
```

Automated tests (mocked models): all passing

```
$ python -m pytest -q tests/test_rag_features.py tests/test_org_isolation.py \
      tests/test_retrieval_first.py tests/test_knowledge_pool.py
........................................................................ [ 96%]
...                                                                      [100%]
75 passed in 30.72s

$ python -m pytest -q          # full suite
337 passed in 45.57s

New this week (28 tests): Unstructured parsing + fallback, semantic breakpoints,
parent-child evidence, corrective retrieval (and never for off-topic questions),
indirect prompt injection, forced re-index, guardrail matrix for the default flow:
  injection x5, small talk x5, overlong input, off-topic x5,
  output check (invented details replaced), invalid citations removed.
```